flutter_application_2/Lab_5png.md

Lab 4: Implementation of a mobile application using an example image recognition model.

Objective

To create a cross-platform application in Flutter that allows taking a photo (or selecting one from the gallery) and classifying the object visible in it using a pre-trained TensorFlow Lite model.

Step 1: Create a new Flutter project: “image_recognition_app”

1. Open Visual Studio Code, type -> , next Flutter -> and click on Create a new Flutter project
2. Enter the project name “image_recognition_app”
3. Click on Create project
4. Select the location where you want to save your project
5. Add to pubspec.yaml file, the required libraries

```
dependencies:
  flutter:
    sdk: flutter
  tflite_flutter: ^0.10.4 # Library for TensorFlow Lite support
  image_picker: ^1.0.4    # Library for picking images from gallery/camera
  image: ^4.1.3           # Library for image manipulation (scaling)
```

6. After adding the dependencies, run:

```
flutter pub get
```

And optionally update the old packages:

```
flutter pub upgrade "package_name"
```

Step 2: Adding the Artificial Intelligence model

7. Find and download the pre-trained model from TensorFlow Lite website or any other source.

- For examle: https://github.com/tflite-soc/tensorflow-models/tree/master/mobilenet-v1
- Folder: asserts/

8. Register the folder in the pubspec.yaml file:

```
# The following section is specific to Flutter packages.
flutter:
  # The following line ensures that the Material Icons font is
  # included with your application, so that you can use the icons in
  # the material Icons class.
  uses-material-design: true
  assets:
    - assets/assets/mobilenet.tflite.  # File with the model
    - assets/labels_mobilenet.txt      # File with the labels
```

Step 3: Implement classification logic

1. Create a `lib/image_classifier.dart` file. We will place the class responsible for loading the model, processing the image, and returning the results here.

```
class ImageClassifier {
  Interpreter? _interpreter;
  List<String>? _labels;

}
```

2. Create the method for loading the labels

```
Future<void> _loadLabels() async {
    final labelData =
      await rootBundle.loadString('assets/labels.txt');
    _labels = labelData.split('\n');
}
```

3. Create the method for loading the model

```
Future<void> loadModel() async {
  try {
    // Loading the TFLite model from the assets folder
    _interpreter = await Interpreter.fromAsset('assets/model.tflite');
    // Loading text labels
    await _loadLabels();
    print('Model loaded successfully.');
  } catch (e) {
    print('Error while loading the model: $e');
  }
}
```

4. Create the method for perform classification of the images

```
Future<String> classifyImage(File imageFile) async {
    if (_interpreter == null || _labels == null) {
      return "Model has not been initialized.";
    }

// implementation of the method

}
```

4.1. Implement the loading of the image

```
final imageData = imageFile.readAsBytesSync();
img.Image? image = img.decodeImage(imageData);
if (image == null) return "Image decoding error.";
```

4.2. Implement the pre-processing: Scaling to 224x224 (MobileNet requirement)

```
img.Image resizedImage = img.copyResize(image, width: 224, height: 224);

// Converting the image to a flat list (for quantized Uint8 model)
var input = List.generate(
  1,
  (i) => List.generate(
    224,
    (j) => List.generate(
      224,
      (k) => List.generate(3, (l) => 0),
    ),
  ),
);

for (int y = 0; y < 224; y++) {
  for (int x = 0; x < 224; x++) {
    final pixel = resizedImage.getPixel(x, y);
    input[0][y][x][0] = pixel.r.toInt(); // Red
    input[0][y][x][1] = pixel.g.toInt(); // Green
    input[0][y][x][2] = pixel.b.toInt(); // Blue
  }
}
```

4.3. Create the output buffer for key for the labels

```
var output = List.filled(1 * 1001, 0).reshape([1, 1001]);
```

4.4. Perform inference of the model

```
_interpreter!.run(input, output);
```

4.5. Implement the post-processing: Finding the index with the highest probability

```
int highestProbabilityIndex = 0;
int highestProbability = 0;

for (int i = 0; i < output[0].length; i++) {
  if (output[0][i] > highestProbability) {
    highestProbability = output[0][i] as int;
    highestProbabilityIndex = i;
  }
}
```

4.6. Finally return the corresponding label

```
if (highestProbabilityIndex < _labels!.length) {
    return "Result: ${_labels![highestProbabilityIndex]}
      (Confidence: $highestProbability)";
  } else {
    return "Object not recognized.";
}
```

Step 4: Implement the main.dart file

```
void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  // This widget is the root of your application.
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Image Classifier',
      theme: ThemeData(
        colorScheme: .fromSeed(seedColor: Colors.deepPurple),
      ),
      home: const RecognitionScreen(),
    );
  }
}
```

Step 6: Implement the recognition screen inside the widgets folder

6.1. Implement the loading of the image

```
class RecognitionScreen extends StatefulWidget {
  const RecognitionScreen({super.key});

  @override
  State<RecognitionScreen> createState() => _RecognitionScreenState();
}

class _RecognitionScreenState extends State<RecognitionScreen> {
  File? _image;
  String _result = "Select an image to classify";
  final ImagePicker _picker = ImagePicker();
  late ImageClassifier _classifier;

// Other parts of the class for implementation

}
```

6.3. Implement the initialization method

```
@override
  void initState() {
    super.initState();
    _classifier = ImageClassifier();
    _classifier.loadModel(); // Initialize model on startup
  }
```

6.4. Implement the function to fetch an image from the gallery

```
Future<void> _pickImage() async {
  final XFile? pickedFile =
    await _picker.pickImage(source: ImageSource.gallery);

  if (pickedFile != null) {
    setState(() {
      _image = File(pickedFile.path);
      _result = "Processing...";
    });
    _classifyImage();
  }
}
```

6.5. Implement the that calls the model and updates the UI state

```
Future<void> _classifyImage() async {
  if (_image == null) return;

  final result = await _classifier.classifyImage(_image!);
  setState(() {
    _result = result;
  });
}
```

6.6. Implement the user interface

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Edge Image Recognition')),
    body: Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          // Displaying the selected image
          _image == null
              ? const Text('No image selected.')
              : Image.file(_image!, height: 300, fit: BoxFit.cover),
          const SizedBox(height: 20),
          // Displaying the AI model result
          Text(
            _result,
            style: const TextStyle(fontSize: 18,
              fontWeight: FontWeight.bold),
            textAlign: TextAlign.center,
          ),
          const SizedBox(height: 20),
          ElevatedButton.icon(
            onPressed: _pickImage,
            icon: const Icon(Icons.photo),
            label: const Text('Select from Gallery'),
          ),
        ],
      ),
    ),
  );
}
```